



Data Structures in Python

Lists, Tuples, Sets and Dictionaries

Data Structures in Python

- Data Structures are a way of storing and organizing data in a computer.

Python has built-in support for several data structures, such as **lists, tuples, dictionaries, and sets**.

- List , Tuple, String are called **Sequence Type** in Python

Lists



- List is a built-in data structure used to store an ordered collection of items. They are dynamic, resizable and capable of storing multiple data types.
- lists are mutable
- **Mutable** - Elements can be reassigned or removed, and new elements can be inserted
- Creating a list is as simple as putting different comma-separated values between square brackets.
- Example : `myList = [1+3, 2.7, 'Thursday']`

Lists



- List is
 - **Mutable**: list elements can be changed, updated, added, or removed after the list is created.
 - **Ordered**: elements maintain the order in which they are inserted.
 - **Index-based**: elements are accessed using their position, starting from index 0.

Lists



■ Creating a List

Lists can be created in several ways, such as using **square brackets []**, the **list()** constructor or by **repeating elements**.

```
a = [1, 2, "HELLO"]  
print(a)                [1, 2, "HELLO"]
```

```
a = list()  
print(a)                 []
```

```
a = [2] * 5  
print(a)                 [2,2,2,2,2]
```

Lists



Accessing Values in List

- To access the i'th element of a list, use the list name followed by the index in square bracket

- Example

```
MyList = ["C", "Python", "Java"]
```

```
# accessing a element from the
```

```
# list using index number
```

```
print("Accessing a element from the list")
```

```
print(MyList[0])
```

```
print(MyList[2])
```

a[3][2] returns 9

Lists



- Lists can contain any kind of Python object, including other lists, as in the following

- example:

```
a = [1, "Dave", 3.14, ["Mark", 7, 9, [100, 101]], 10]
print(a[3][0])      # it prints "Mark"
```

- An empty list is created in one of two ways:

```
names = []
names = list()
```

Lists



Updating Elements into List

- Since lists are mutable, elements can be updated by assigning new values using their index.
- Example:

```
a = [10, 20, 30, 40, 50]  
a[1] = 25  
print(a)
```


Basic Operations on Sequences

The following operators can be applied to sequence types, including strings, lists, and tuples:

Operation	Description
<code>s + r</code>	Concatenation
<code>s * n, n * s</code>	Makes <i>n</i> copies of <i>s</i> , where <i>n</i> is an integer
<code>v1, v2..., vn = s</code>	Variable unpacking
<code>s[i]</code>	Indexing
<code>s[i:j]</code>	Slicing
<code>s[i:j:stride]</code>	Extended slicing
<code>x in s, x not in s</code>	Membership
<code>for x in s:</code>	Iteration
<code>all(s)</code>	Returns True if all items in <i>s</i> are true.
<code>any(s)</code>	Returns True if any item in <i>s</i> is true.
<code>len(s)</code>	Length
<code>min(s)</code>	Minimum item in <i>s</i>
<code>max(s)</code>	Maximum item in <i>s</i>
<code>sum(s [, initial])</code>	Sum of items with an optional initial value

Basic Operations on Sequences

```
>>> a = [3,4,5]
>>> b = [a]
>>> c = 4*a
>>> c
[3, 4, 5,3, 4, 5,3, 4, 5,3, 4, 5]

>>d = 3*b
>>d
[[3.4.5],[3.4.5],[3.4.5]]
```

Variable unpacking

```
items = [ 3, 4, 5 ]
x,y,z = items # x = 3, y = 4, z = 5
letters = "abc"
x,y,z = letters # x = 'a', y = 'b', z = 'c'
```

Slicing in List

- The indexing operator **s[n]** returns the n^{th} object from a sequence
- Negative indices can be used to fetch characters from the end of a sequence
- The slicing operator **s[i:j]** extracts a subsequence from s consisting of the elements with index k , where $i \leq k < j$
- **s[i:j:stride]** , that causes the slice to skip elements

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

b = a[::2]	# b = [0, 2, 4, 6, 8]
c = a[::-2]	# c = [9, 7, 5, 3, 1]
d = a[0:5:2]	# d = [0,2]
e = a[5:0:-2]	# e = [5,3,1]
f = a[:5:1]	# f = [0,1,2,3,4]
g = a[:5:-1]	# g = [9,8,7,6]
h = a[5::1]	# h = [5,6,7,8,9]
i = a[5::-1]	# i = [5,4,3,2,1,0]
j = a[5:0:-1]	# j = [5,4,3,2,1]

Slicing in List

Strings and tuples are immutable and cannot be modified after creation. Lists can be modified with the following operators:

Operation	Description
<code>s[i] = x</code>	Index assignment
<code>s[i:j] = r</code>	Slice assignment
<code>s[i:j:stride] = r</code>	Extended slice assignment
<code>del s[i]</code>	Deletes an element
<code>del s[i:j]</code>	Deletes a slice
<code>del s[i:j:stride]</code>	Deletes an extended slice

Slicing in List



- **extract or reassign** a portion of a list by using the slicing operator:

```
names = ["Jeff", "Mark", "Thomas", "Ann", "Phil", "Paula"]
```

- `b = names[0:2]` `#Returns [ "Jeff", "Mark" ]`
- `c = names[2:]` `#Returns ["Thomas", "Ann", "Phil", "Paula"]`
- `names[1] = 'Jeff'` `#Replace the 2nd item in names with 'Jeff'`
- `names[0:2] = ['Dave', 'Mark']` `# Replace the first two items of the list with the list on the right.`

Lists - functions



Python has set of built-in methods that you can use of list/arrays

Method	Description
<u>append()</u>	Adds an element at the end of the list
<u>clear()</u>	Removes all the elements from the list
<u>copy()</u>	Returns a copy of the list
<u>count()</u>	Returns the number of elements with the specified value
<u>extend()</u>	Add the elements of a list (or any iterable), to the end of the current list
<u>index()</u>	Returns the index of the first element with the specified value
<u>insert()</u>	Adds an element at the specified position
<u>pop()</u>	Removes the element at the specified position
<u>remove()</u>	Removes the first item with the specified value
<u>reverse()</u>	Reverses the order of the list
<u>sort()</u>	Sorts the list

append

Lists - functions

- To **append** new items to the end of a list, use the **append()** method:

- Syntax

list.append(obj)

- Parameters

obj – This is the object to be appended in the list.

- Return Value

This method does not return any value but updates existing list.

Example:

- `names=list()`
- `names.append("Paula")`

insert

Lists - functions

- To insert an item into the middle of a list, use the **insert()** method:
- **Syntax**
`list.insert(index, obj)`
- **Parameters**
 - `index` – Index where the object `obj` need to be inserted.
 - `obj` – Object to be inserted into the given list.
- **Return Value**
This method does not return any value but it inserts the given element at the given index.

Example

- `names.insert(2, "Thomas")`

reverse

Lists - functions

- Python list method reverse() reverses objects of list in place.

- Syntax**

`list.reverse()`

- Parameters**

NA

- Return Value**

This method does not return any value but reverse the given object from the list.

Example

```
aList = [123, 'xyz', 'zara', 'abc', 'xyz'];  
aList.reverse();
```

- Python list method `sort()` sorts objects of list

- **Syntax**

`list.sort(reverse=True|False)`

- **Parameters**

optional

- **Return Value**

This method does not return any value but it changes from the original list.

Example

```
aList=[4,7,1,3,9]  
aList.sort();
```

The list `copy()` function is used to create a copy of a list, which can be used to work and it will not affect the values in the original list. This gives freedom to manipulate data without worrying about data loss.

- **Syntax**

`list.copy()`

- **Parameters**

NA

- **Return Value**

This method returns a copy of the specified list.

Example

```
fruits = ['apple', 'banana', 'cherry', 'orange']  
x = fruits.copy()
```

- Python list method `count()` returns count of how many times `obj` occurs in list.
- **Syntax**
`list.count(obj)`
- **Parameters**
`obj` – This is the object to be counted in the list.
- **Return Value**
This method returns count of how many times `obj` occurs in list.

Example

```
aList=[4,7,4,1,3,4,9]  
print("count for 4 :", aList.count(4))
```

- Python list method `index()` returns the lowest index in list that `obj` appears.
- **Syntax**
`list.index(obj)`
- **Parameters**
`obj` – This is the object to be find out.
- **Return Value**
This method returns index of the found object otherwise raise an exception indicating that value does not find.

Example

```
aList = [123, 'xyz', 'zara', 'abc'];  
print("Index for xyz : ", aList.index( 'xyz' ))
```

remove

Lists - functions

- Python list method **remove()** searches for the given element in the list and removes the first matching element.:
- **Syntax**
`list.remove(obj)`
- **Parameters**
`obj` – This is the object to be removed from the list.
- **Return Value**
This Python list method does not return any value but removes the given object from the list.

Example

```
aList.remove('xyz');
```



pop

Lists - functions



Removes the element at the specified position

- **Syntax**

`list.pop(pos)`

- **Parameters**

`pos` - Optional. A number specifying the position of the element you want to remove, default value is -1, which returns the last item

Example

```
List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]  
print(List.pop(0))
```



del

Lists - functions

del statement - Element to be deleted is mentioned using list name and index.

- **Syntax**

```
del list[index]
```

- **Example**

```
my_list = [2.3, 4.445, 3, 5.33, 1.054, 2.5]
```

```
del my_list[0]
```

```
# deleting items from 2nd to 4th
```

```
del my_list[1:4]
```

```
print(my_list)
```


sum

Lists - functions



`sum()` : Calculates sum of all the elements of List.

- **Syntax**

`sum(List)`

Example

```
List = [1, 2, 3, 4, 5]  
print(sum(List))
```



min

Lists - functions



min() : Calculates minimum of all the elements of List.

Syntax:

`min(List)`

max() : Calculates maximum of all the elements of List.

Syntax:

`max(List)`

▪ Example

`List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]`

`print(max(List))`

`print(min(List))`

The Python List `extend()` method adds items of an iterable (list, tuple, dictionary, etc) at the end of a list.

Syntax:

```
list.extend(iterable)
```

Parameter Required. Any iterable (list, set, tuple, etc.)

■ Example

```
fruits = ['apple', 'banana', 'cherry']  
cars = ['Ford', 'BMW', 'Volvo']  
fruits.extend(cars)
```